

Infinite Connections

An AI System for Generating NYT-Style Connections Puzzles Executive Summary of Current Results and Implementation Pipelines

STA 561D | Duke University | April 2026

1. Project Overview

The New York Times Connections puzzle presents players with 16 words that must be partitioned into 4 groups of 4 based on a hidden theme. Infinite Connections is a system that generates such puzzles automatically at scale, validates them with multiple independent solvers, and serves them through a playable web interface.

We have built and benchmarked two complete pipelines:

1. Pipeline A -- LLM-Heavy (baseline): 5 LLM calls per puzzle; LLM proposes word pools, MPNET embeddings refine selection.
2. Pipeline B -- Category-First Retrieval (CFR, novel): 0 or 1 LLM calls per puzzle; embeddings drive word retrieval directly from the 4,918-word NYT bank.

In addition, we have deployed a playable web application hosted via GitHub Pages that loads 542 validated puzzles and replicates the NYT Connections UI.

Headline result (v2): Pipeline B (CFR) achieves a 99% validator pass rate while drawing 62-69% of its output words from non-NYT sources, reproducing ZERO past NYT groups across 200 benchmark puzzles, and running at 1/20 the cost and 4x the speed of the baseline LLM-heavy pipeline.

CFR v2 (rubric-compliant upgrade)

A careful re-read of the rubric surfaced two constraints the first-pass CFR did not meet. We fixed both:

- "If you generate a past connections puzzle you will automatically fail." -- Added an exact 4-word-group match check against all ~2,216 past NYT groups; if the top-scoring retrieval would reproduce an NYT group, the next-best 4-word combination is used instead.
- Professor guidance: the word bank should be larger than the original game's. -- Augmented the 4,918-word NYT bank with 10,000+ common English lemmas from WordNet, yielding a 14,877-word bank (3x larger than NYT's).

2. Pipeline A -- LLM-Heavy Baseline

Pipeline A follows the architecture from "Making New Connections" (arXiv:2407.11240). For each puzzle it issues 5 LLM calls:

1. Calls 1-4 (Group creation). Four iterative calls, one per group. Each returns a category name plus 8 candidate words. Story injection (4 random seed words from the NYT bank) forces diversity.
2. MPNET selection (non-LLM). Of the 8 candidates, the best 4 are chosen by enumerating all $C(8,4)=70$ subsets and picking the one with highest average pairwise cosine similarity.
3. Call 5 (Editor pass). A final LLM call reviews all 4 category names and rewrites any that don't accurately describe their words.
4. Difficulty color assignment (non-LLM). The 4 groups are sorted by avg pairwise cosine similarity; highest -> yellow, lowest -> purple.
5. Deduplication (non-LLM). The 16-word set is checked against all 554 NYT puzzles; flagged if >6 words overlap

with any single puzzle.

6. Roundtable validation (non-LLM). Embedding solver and beam-search clustering solver attempt to recover the intended grouping. A puzzle is accepted if either solver fully solves it.

Pipeline A Results

Batch	Model	Count	Valid	Pass rate
100-puzzle (mini)	gpt-4o-mini	100	91	91%
100-puzzle (4o)	gpt-4o	100	87	87%
575-puzzle batch	gpt-4o-mini	575	534	93%
TOTAL		775	712	91.9%

Observation: gpt-4o produces more creative wordplay categories but lower pass rate (87%) because embedding-based validators struggle with pure wordplay. gpt-4o-mini is a better fit for this pipeline.

3. Pipeline B -- Category-First Retrieval (CFR)

Key insight: in Pipeline A, the LLM wastes tokens generating candidate words, only to have MPNET pick 4 of them anyway. CFR flips the flow:

- Propose 4 categories (two modes):
 - Mode A (remix): sample 4 mutually dissimilar categories from the 2,054 existing NYT categories (diversity enforced via pairwise MPNET cosine distance ≥ 0.35). Zero LLM calls.
 - Mode B (fresh): one batched LLM call returns 4 fresh category names in a single response.
- Retrieve words per category. For each category name, encode with MPNET and run KNN over the precomputed 4,918-word embedding matrix to get the top-30 semantically closest words. Filter morphological duplicates (TIME/TIMES/TIMER) and category-token overlaps ("TIME" for category "TIME PERIODS").
- Select best 4. Reuse Pipeline A's MPNET selector to pick the 4 most cohesive words from the top 8 candidates.
- Assign colors, deduplicate, validate. Identical to Pipeline A.

Offline precomputation (one-time, ~5 minutes)

- Encode all 4,918 NYT words -> 4918 x 768 matrix.
- Encode all 2,054 unique NYT categories -> 2054 x 768 matrix.
- Build a scikit-learn NearestNeighbors index (cosine distance).
- Cache the result as data/cache/nyt_embeddings.npz.

Pipeline B Results (v1 vs. v2)

Version & Mode	Word bank	Pass	Non-NYT words	NYT group collisions	Time
v1 -- Mode A (remix)	4,918	97%	0%	not checked	1.41 s
v1 -- Mode B (fresh)	4,918	99%	0%	not checked	2.67 s
v2 -- Mode A (remix)	14,877	99%	62.6%	0 / 100	1.27 s
v2 -- Mode B (fresh)	14,877	99%	69.1%	0 / 100	2.77 s

4. Head-to-Head Comparison

Metric	Pipeline A	CFR v2 Mode A	CFR v2 Mode B
--------	------------	---------------	---------------

LLM calls / puzzle	5	0	1
Cost / 1,000 puzzles	~\$1.00	\$0.00	\$0.05
Wall time / puzzle	~10 s	1.27 s	2.77 s
Pass rate	87-93%	99%	99%
Word bank size	4,918 (NYT)	14,877	14,877
Non-NYT words in output	0%	62.6%	69.1%
Hard-fail safety check	16-word	+ 4-word group	+ 4-word group
NYT group collisions (100)	not checked	0	0
API fragility	High	None	Low

Speed

CFR Mode A is ~7x faster and Mode B ~4x faster than the baseline, because KNN retrieval on 4,918 words is essentially instantaneous compared to network round-trips to an LLM API.

Cost

For a run of 10,000 puzzles, the baseline costs ~\$10; CFR Mode A costs \$0 and Mode B ~\$0.50 -- a 20x cost reduction at higher quality.

Methodological contribution

The project's original contribution shifts from "we called the LLM to generate puzzles" to "we built an embedding-first hybrid pipeline where the LLM is a small, targeted component."

5. Rubric-Compliance Upgrade (CFR v1 -> v2)

The original CFR worked well by statistical metrics but missed two rubric-level constraints. The v2 upgrade keeps the architecture identical and adds two targeted safeguards.

Fix 1: Augmented word bank (new src/cfr/word_bank.py)

The professor noted that "the total word bank should be more than that of the original game's." We union three sources and deduplicate:

1. The 4,918 unique NYT words (retained for NYT-style familiarity).
2. NLTK WordNet single-token lemmas with nonzero tagged-corpus frequency -- filtered to 3-15 chars, alphabetic only, no profanity. Yields ~10,000 common English words (no archaic/technical noise).
3. Optional Google 10k most-common-English list if present (not required).

Result: 14,877 words, cached as data/cache/augmented_word_bank.json. The KNN retriever is unchanged; it just searches over a bigger universe.

Fix 2: 4-word-group hard-fail guard (extended Deduplicator)

The rubric says: "If you generate a past connections puzzle you will automatically fail." The original dedup only flagged 16-word-set overlap > 6 against past NYT puzzles. We added:

- Precomputed _nyt_group_set of ~2,216 past NYT 4-word groups as frozensets, indexed at first call.
- New check_groups(groups) method that flags exact 4-word matches against any past NYT group.
- In CFR's retrieval, we enumerate all $C(8,4)=70$ combinations of candidate words, rank by avg pairwise cosine

similarity, and return the first combination that is NOT a verbatim NYT group.

Across 200 v2 puzzles (100 Mode A + 100 Mode B), ZERO accidentally reproduced a past NYT group.

Verified impact (v1 -> v2)

Metric	v1	v2
Word bank size	4,918	14,877
4-word group dedup	X	YES
Non-NYT words in output (Mode A)	0%	62.6%
Non-NYT words in output (Mode B)	0%	69.1%
Exact NYT group matches (200 puzzles)	unknown	0
Pass rate Mode A	97%	99%
Pass rate Mode B	99%	99%

6. Solver Benchmark (Sanity Check on NYT Data)

To verify the validator is not over-accepting our own puzzles, we ran both solvers on the 554 ground-truth NYT puzzles:

Solver	Full-puzzle solve rate	Paper baseline
Embedding solver (greedy)	2.0%	~11.6% (Missed Connections)
Clustering solver (beam)	3.4%	higher than embedding

Our solvers correctly find only 2-3% of real NYT puzzles (which often contain pop culture / wordplay categories that defeat semantic similarity). This is the right kind of conservative validator -- a puzzle that our solvers can crack deterministically is one where the semantic structure is cleanly recoverable, i.e., a solvable puzzle.

7. Deployed Artifacts

- 542-puzzle static web app: live on GitHub Pages at <https://kevin2330.github.io/nyt-connections-generator/>. Vanilla JS frontend; client-side puzzle selection; no backend required.
- Color-split dataset: 534 validated groups in each of 4 difficulty buckets (yellow / green / blue / purple), totaling 8,544 NYT-style word groups.
- Jupyter notebook (notebooks/infinite_connections_demo.ipynb) reproducing all generation and validation steps in dry-run mode.
- CFR module (src/cfr/): 3 new Python files (~300 lines total), completely separate from the baseline pipeline. All existing code is untouched.

8. Summary of Current Results

Artifact	Value
Total puzzles generated	1,175+ (775 baseline + 400 CFR)
Total validated puzzles	1,106+ (712 baseline + 394 CFR)
Best pass rate	99% (CFR v2, both modes)
Fastest per-puzzle generation	1.27 s (CFR v2 Mode A, 0 LLM calls)

Total spending to date	~\$2.50
Word bank (v2)	14,877 words (4,918 NYT + ~10,000 WordNet)
Non-NYT word usage (v2)	62-69% of output words
NYT group collisions (v2)	0 across 200 puzzles (rubric safety)
Web app	Deployed, 542 puzzles playable online
Pipelines built	2 novel (CFR Mode A + Mode B)
Methodological novelty	Embedding-first retrieval + group dedup

Key Takeaway

The project has produced both a competent LLM-heavy baseline AND a new, less-LLM-reliant methodology (CFR) that beats the baseline on every measurable axis: pass rate, cost, and speed. CFR makes the academic contribution meaningfully original: we don't just wrap an LLM, we use embeddings as the primary generative substrate and let the LLM play a small, well-defined role (writing 4 category names). The system is fully reproducible, deployed, and documented.